An Analytical Performance Model for Streaming Tensor Programs

1 Overview

We present an analytical timing model for dataflow graphs expressed in the Streaming Tensor Programs (STeP) abstraction. The model captures the steady-state throughput and startup latency of each operator in a STeP graph, accounting for compute-bound and memory-bound regimes via Roofline modeling. Dynamic tensor dimensions propagate as symbolic variables, enabling closed-form analysis that can be instantiated with concrete traces, distributions, or worst-case bounds.

The model structure is inspired by the Stream-HLS performance model [1], adapted to STeP's operator-based dataflow semantics. The key reformulation replaces loop-nest analysis with a rate-based abstraction built on chunk intervals and tile counts, which naturally captures the multi-rate behavior of STeP operators.

2 Graph Structure

A STeP program is modeled as a directed acyclic graph (DAG) G. Each STeP operator is a node; each stream connecting a producer to a consumer is an edge. We process nodes in topological order. The total execution time of the program is:

$$T_{\text{graph}} = \max_{n \in \text{leaf}(G)} \big(end(n)\big) \tag{1}$$

3 Per-Node Quantities

For each node n in the graph, we define the following quantities, derived from the operator type, its stream shapes, and the hardware parameters.

Symbol	Name	Description
$T_{\text{fire}}(n)$	Firing time	Processing time per firing (Roofline)
$N_{\text{fire}}(n)$	Firing count	Total number of firings; may be symbolic
$NIT_n^{n'}$	Input tile count	Input tiles consumed from predecessor n' per firing
$OTPC(n)$	Output tiles per chunk	Output tiles produced per firing

Table 1: Per-node quantities defined by operator type and stream shapes.

The firing time is given by the Roofline equation:

$$T_{\text{fire}}(n) = \max\left(\frac{Size_{\text{in}}}{BW_m}, \frac{FLOPs}{BW_c}, \frac{Size_{\text{out}}}{BW_m}\right) \tag{2}$$

where $Size_{\text{in}}$ and $Size_{\text{out}}$ are the total input and output data sizes per firing, $FLOPs$ is the arithmetic work per firing, BW_c is the allocated compute bandwidth, and BW_m is the on-chip memory bandwidth. When inputs and outputs are streamed directly via hardware FIFOs (no scratchpad access), the memory bandwidth terms drop to zero. For structural operators with no compute (shape operators, routing operators, Bufferize), $T_{\text{fire}}(n) = 0$.

4 Core Timing Equations

We compute the following quantities for each node n in topological order.

4.1 Start Time

A node cannot begin execution until all its input streams carry data. For source nodes (no predecessors), $st(n) = 0$.

$$st(n) = \max_{n' \in \text{pred}(n)} \big(Arrives(n, n') \big) \tag{3}$$

where:

$$Arrives(n, n') = \begin{cases} fto(n') & \text{direct stream edge} \\ fto(n') + L_{\text{HBM}} & \text{off-chip edge} \end{cases} \tag{4}$$

For a direct stream edge, the consumer can start as soon as the producer emits its first output tile. For an off-chip edge (where data passes through HBM between a `LinearOffChipStore` and a `LinearOffChipLoad`), the HBM access latency L_{HBM} is added.

4.2 First Tile Out

The absolute time at which node n produces its first output tile. Before producing output, the node must (1) wait for enough input tiles to arrive to complete its first firing, and (2) perform one firing of computation.

$$fto(n) = st(n) + FTO_n \tag{5}$$

where FTO_n is the *relative* time of the first tile out:

$$FTO_n = ICD(n) + T_{\text{fire}}(n) \tag{6}$$

$ICD(n)$ is the *input chunk delay*: the time from when the first input tile arrives until the full input chunk for the first firing is available. It captures the wait for the remaining $NIT - 1$ tiles to arrive at each predecessor's output rate:

$$ICD(n) = \max_{n' \in \text{pred}(n)} \big((NIT_n^{n'} - 1) \cdot OTI(n') \big) \tag{7}$$

4.3 Last Tile Out

The absolute time at which node n produces its last output tile:

$$lto(n) = fto(n) + (OTPC(n) - 1) \cdot OTI(n) \tag{8}$$

Note: for operators with $OTPC(n) = 1$ (most operators), $lto(n) = fto(n)$ on the first firing. The distinction matters for operators like `FlatMap` and `Streamify` that emit multiple tiles per firing.

4.4 End Time

The absolute time at which node n completes all its firings:

$$end(n) = st(n) + N_{\text{fire}}(n) \cdot OCI(n) \tag{9}$$

4.5 Output Chunk Interval (OCI)

The time between consecutive output chunks produced by node n. The node's output rate is limited by either its own compute time or the rate at which its input chunks arrive, whichever is slower:

$$OCI(n) = \max\big(T_{\text{fire}}(n),\ ICI(n) \big) \tag{10}$$

This equation captures backpressure: if inputs arrive slower than the node can process them, the node is input-starved and OCI is dominated by ICI. If the node's own compute is the bottleneck, OCI is dominated by T_{fire}.

4.6 Input Chunk Interval (ICI)

The time between consecutive input chunks becoming available for node n. Each firing of n consumes $NIT_n^{n'}$ tiles from each predecessor n', so the interval is determined by how long it takes each predecessor to produce that many tiles:

$$ICI(n) = \max_{n' \in \text{pred}(n)} \left(NIT_n^{n'} \cdot OTI(n') \right) \tag{11}$$

4.7 Output Tile Interval (OTI)

The time between consecutive output tiles produced by node n. For compute and on-chip memory operators, if each firing produces $OTPC(n)$ tiles, the per-tile interval is:

$$OTI(n) = \frac{OCI(n)}{OTPC(n)} \tag{12}$$

This converts from the chunk-level rate to the tile-level rate, which is needed by downstream nodes to compute their own ICI and ICD.

For off-chip memory operators, the OTI is instead determined by memory system contention, as described in Section 5.

4.8 Summary of Dependencies

The equations form a chain that propagates rates through the graph in topological order. For each node, the computation order is:

$$OTI(\text{predecessors}) \;\rightarrow\; ICI(n) \;\rightarrow\; OCI(n) \;\rightarrow\; OTI(n)$$

In parallel, the startup path is:

$$OTI(\text{predecessors}) \;\rightarrow\; ICD(n) \;\rightarrow\; FTO_n \;\rightarrow\; fto(n) \;\rightarrow\; st(\text{successors})$$

The total graph latency combines both: a startup transient (accumulated through the fto/st chain) plus a steady-state execution phase ($N_{\text{fire}} \cdot OCI$ at each node).

5 Memory Contention Model

When multiple off-chip memory operators are active concurrently, they compete for a shared set of memory channels. We model this contention by computing the output tile interval of off-chip operators directly from the memory system parameters rather than from the Roofline equation.

For any off-chip memory operator n:

$$OTI(n) = \max\left(1,\ \frac{R_{\text{total}}}{C}\right) \tag{13}$$

where R_{total} is the aggregate number of steady-state memory requests per cycle across *all* concurrently active off-chip operators, and C is the number of memory channels.

The $\max(1, \cdot)$ ensures that even when the memory system is underutilized (fewer requests than channels), the minimum interval is 1 cycle per tile. When the memory system is oversubscribed ($R_{\text{total}} > C$), the interval stretches proportionally. All concurrently active off-chip operators see the same OTI, since they share the same congested memory system. This contention then propagates through the ICI/OCI chain to downstream compute nodes.

Determining concurrent activity. Precisely identifying which off-chip operators are concurrently active requires knowing their time intervals, which in turn depend on OTI. A conservative approach assumes all off-chip operators in the graph are concurrent, yielding an upper bound on contention. A tighter estimate can be obtained via fixed-point iteration or by topological analysis of the graph to identify operators whose $[st, end]$ intervals overlap.

6 Per-Operator Parameters

The tables below define T_{fire}, N_{fire}, NIT, and $OTPC$ for each STeP operator category. Stream shapes are written as $[D_a, \ldots, D_0]$. Symbolic variables (e.g. D_i) propagate through the model.

6.1 Shape Operators (Flatten, Promote, Reshape, Expand, Zip)

Zero-cost stop-token manipulation. These act as pass-through for timing purposes.

Quantity	Value
T_{fire}	0
N_{fire}	$\prod D_i$ (pass-through)
NIT	1
$OTPC$	1

6.2 Map

Element-wise computation. One input element produces one output element. For two-input Map, both inputs are consumed in lockstep.

Quantity	Value
T_{fire}	$\max\left(\frac{S_{\text{in}}}{BW_m}, \frac{F}{BW_c}, \frac{S_{\text{out}}}{BW_m}\right)$
N_{fire}	$\prod D_i$
$NIT_n^{n'}$	1 (for each input stream)
$OTPC$	1

6.3 Accum

Reduction over the inner b dimensions. Input shape $[D_a, \ldots, D_b, D_{b-1}, \ldots, D_0]$. Let $K = \prod_{i=0}^{b-1} D_i$ (elements per reduction) and $M = \prod_{i=b}^{a} D_i$ (number of outputs). K may be symbolic.

Quantity	Value
T_{fire}	$\max\left(\frac{S_{\text{in}}}{BW_m}, \frac{F}{BW_c}\right)$
N_{fire}	M
$NIT_n^{n'}$	K (consumes K input tiles per output)
$OTPC$	1

6.4 Scan

Identical to Map in timing: one output per input. Internal accumulator state does not affect the production rate.

6.5 FlatMap

Each input element produces a sub-stream of E output elements. Let $E = \prod D'_i$ (output elements per input).

Quantity	Value
T_{fire}	$\max\left(\frac{S_{\text{in}}}{BW_m}, \frac{F}{BW_c}, \frac{S_{\text{out}}}{BW_m}\right)$
N_{fire}	$\prod D_i$
$NIT_n^{n'}$	1
$OTPC$	E

6.6 Bufferize

Accumulates $K = \prod_{i=0}^{b-1} D_i$ input tiles, emits one buffer reference. Scratchpad writes are pipelined with input arrival.

Quantity	Value
T_{fire}	0
N_{fire}	$\prod_{i=b}^{a} D_i$ (number of buffers emitted)
$NIT_n^{n'}$	K (tiles accumulated per buffer)
$OTPC$	1

6.7 Streamify

Reads tiles from an on-chip buffer. Let R be the number of reference elements and E be the number of tiles read per buffer.

Quantity	Value
T_{fire}	$S_{\text{tile}}/BW_{\text{scratchpad}}$
N_{fire}	R
$NIT_n^{n'}$	1 (one buffer reference per firing)
$OTPC$	E (tiles emitted per buffer read)

6.8 LinearOffChipLoad

Loads tiles from HBM. Stream shape $[D_a, \ldots, D_0]$. The OTI is determined by the memory contention model (Section 5) rather than the Roofline equation.

6.9 LinearOffChipStore

Stores tiles to HBM. Symmetric to LinearOffChipLoad; OTI is determined by the memory contention model.

6.10 Routing and Merging (Partition, Reassemble, EagerMerge)

Data-dependent routing with negligible processing cost. Dynamic volume per branch (D_i) appears as symbolic variables in the N_{fire} of downstream nodes.

For **Reassemble**, the start time $st(n)$ depends on *fto* of each selected input branch, handled by the core equations. For **EagerMerge**, the output ordering is nondeterministic (arrival-order); the model treats it identically, with $st(n)$ determined by the first branch to produce data and $end(n)$ by the last.

Quantity	Value
T_{fire}	N/A (OTI from contention model)
N_{fire}	$\prod D_i$
$NIT_n^{n'}$	1 (one reference element per tile load)
$OTPC$	1

Quantity	Value
T_{fire}	N/A (OTI from contention model)
N_{fire}	$\prod D_i$
$NIT_n^{n'}$	1
$OTPC$	1

7 Handling Dynamic Dimensions

STeP streams may have dynamic-regular or ragged dimensions. These appear as symbolic variables throughout the model (e.g., D_i for the number of tokens routed to expert i, or K for a dynamic reduction dimension in Accum). The timing equations remain closed-form expressions over these symbols. Three evaluation strategies are available:

Concrete instantiation. Substitute symbols with values from a specific trace (e.g., expert routing data) to obtain a deterministic cycle count.

Distributional analysis. Treat symbols as random variables with known distributions to compute expected latency or tail bounds.

Worst/best-case bounds. Substitute extreme values to obtain performance envelopes.

8 Relationship to the Stream-HLS Model

The model shares the same DAG-based, topological-order structure as Stream-HLS [1]. The key differences are:

(i) **Rate-based reformulation.** Stream-HLS tracks FW_n, LW_n, and $LR_n^{n'}$ as absolute relative times, with a Depend/Epilogue decomposition to handle stalls. This model replaces that with a rate-based chain ($ICD \to ICI \to OCI \to OTI$) that separates startup transients from steady-state throughput. Stall behavior is captured by $OCI(n) = \max(T_{\text{fire}}(n), ICI(n))$.

(ii) **Uniform streaming edges.** Stream-HLS distinguishes FIFO edges ($WAF = RAF$) from shared-buffer edges. In this model, all edges are streaming. Buffering delays emerge from Bufferize nodes (which have $NIT = K$, causing a proportional ICD at downstream consumers) rather than from edge classification.

(iii) **Roofline-based firing times.** Per-node timing is derived from the Roofline model rather than from loop-nest initiation intervals.

(iv) **Memory contention.** Off-chip memory operators use a contention-aware OTI based on aggregate memory requests and channel count, rather than per-operator bandwidth assumptions.

(v) **Symbolic dimensions.** Node parameters may contain symbolic variables representing data-dependent stream dimensions, whereas Stream-HLS operates on statically-known loop bounds.

Quantity	Value
T_{fire}	0
N_{fire}	determined by input/output stream volume
$NIT_n^{n'}$	1
$OTPC$	1

References

[1] S. Basalama and J. Cong, "Stream-HLS: Towards Automatic Dataflow Acceleration," in *Proc. ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25)*, Monterey, CA, USA, Feb. 2025, pp. 103–114.

[2] G. Sohn, G. Zhang, K. Hossfeld, J. Kim, N. Sobotka, N. Zhang, O. Hsu, and K. Olukotun, "Streaming Tensor Programs: A Streaming Abstraction for Dynamic Parallelism," in *Proc. 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '26)*, Pittsburgh, PA, USA, Mar. 2026.